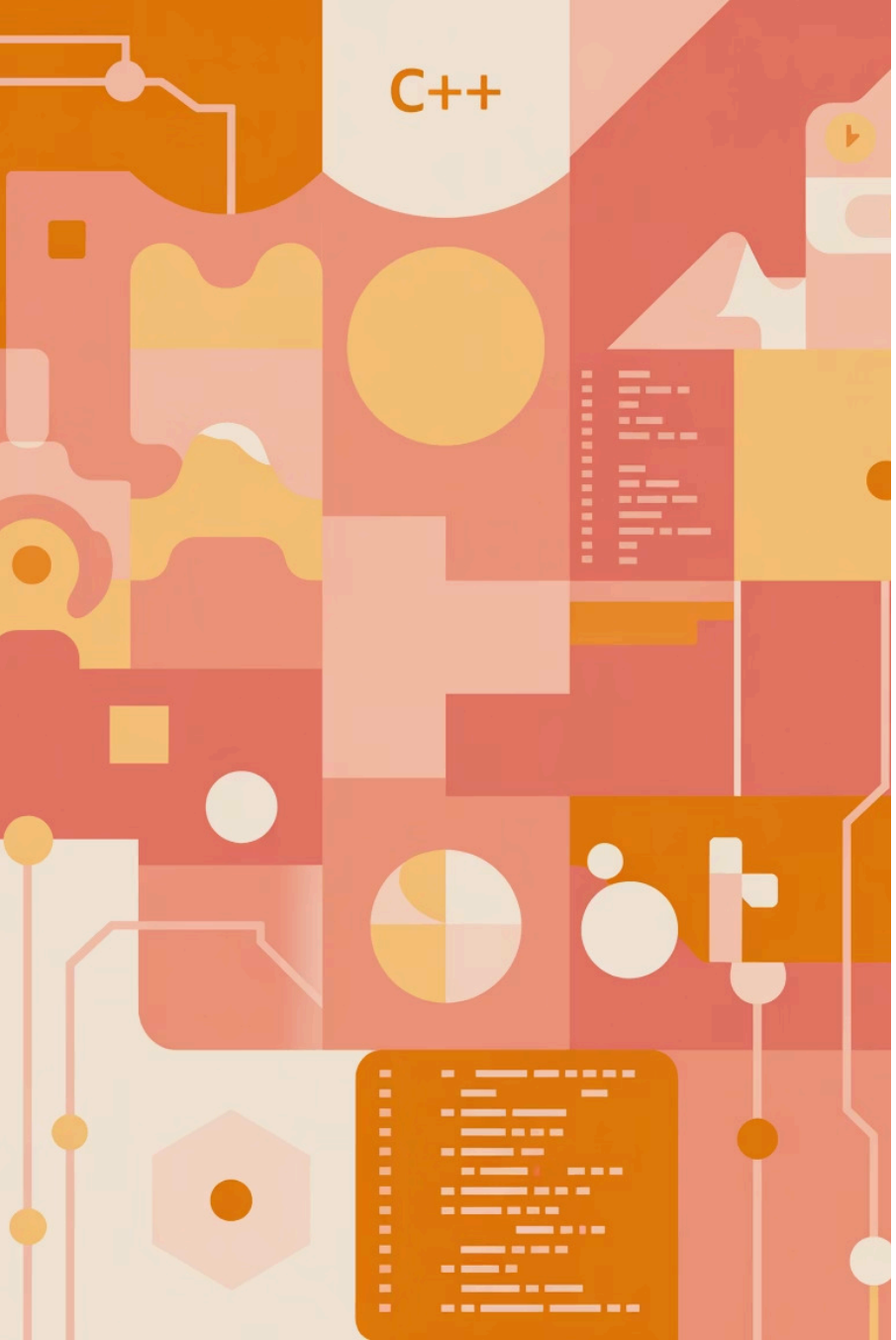


现代C++的发展道路

从哲学到实践

第三篇：RAII —— 让资源管理自动、可靠、无GC

一门语言的长青之道

为什么需要 RAII

> Resource Acquisition Is Initialization

C++ 追求零成本抽象

为此选择不内建垃圾回收（GC）

没有 GC，资源管理仍必须可靠

RAII 是 C++ 给出的核心答案

> RAII 在不引入运行时成本的前提下，提供资源安全。

RAII 的基本思想

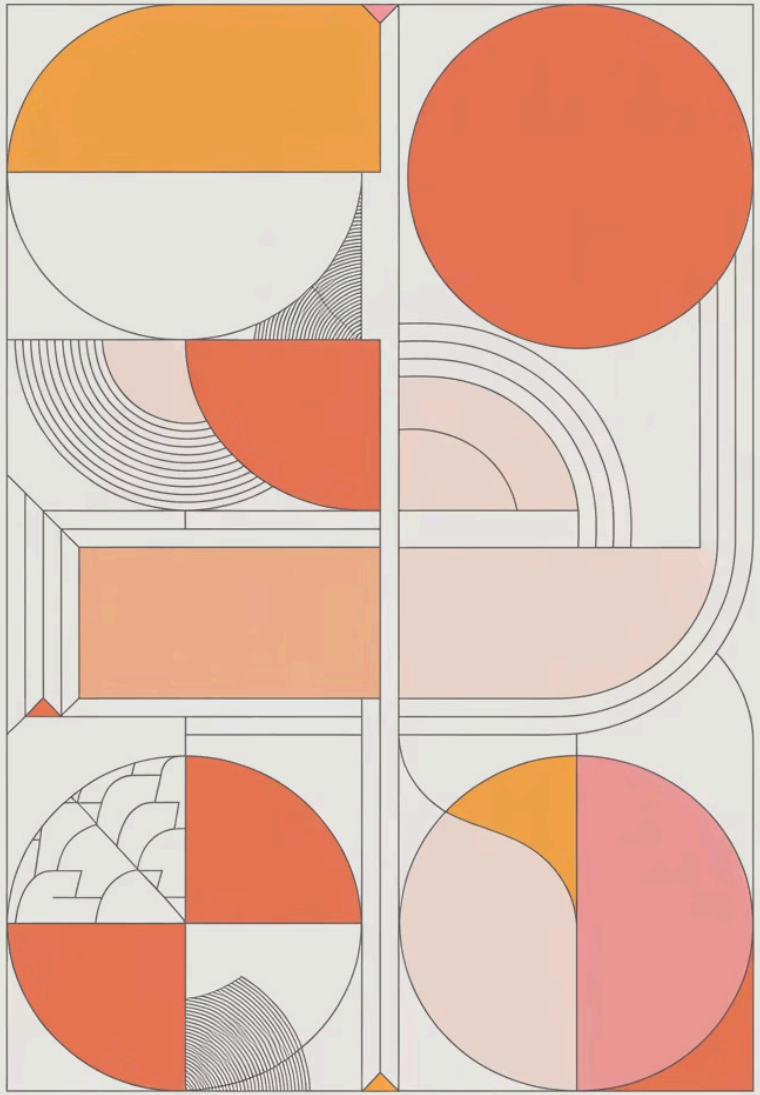
资源由对象管理，而不是由代码路径管理

资源获取发生在构造函数中

资源释放发生在析构函数中

作用域决定对象生命周期

> 进入作用域获取资源，离开作用域自动释放。（右花括号原则）



Ownership：所有权即责任

所有权描述的是资源生命周期责任

独占所有权：可转让，不可复制

共享所有权：可复制，通常依赖引用计数

工程中独占所有权更常见

> Ownership 是一种责任，不是"能用资源"，而是"负责释放资源"。

标准库中的 RAII

std::lock_guard

锁管理

std::unique_ptr / std::shared_ptr

内存与所有权

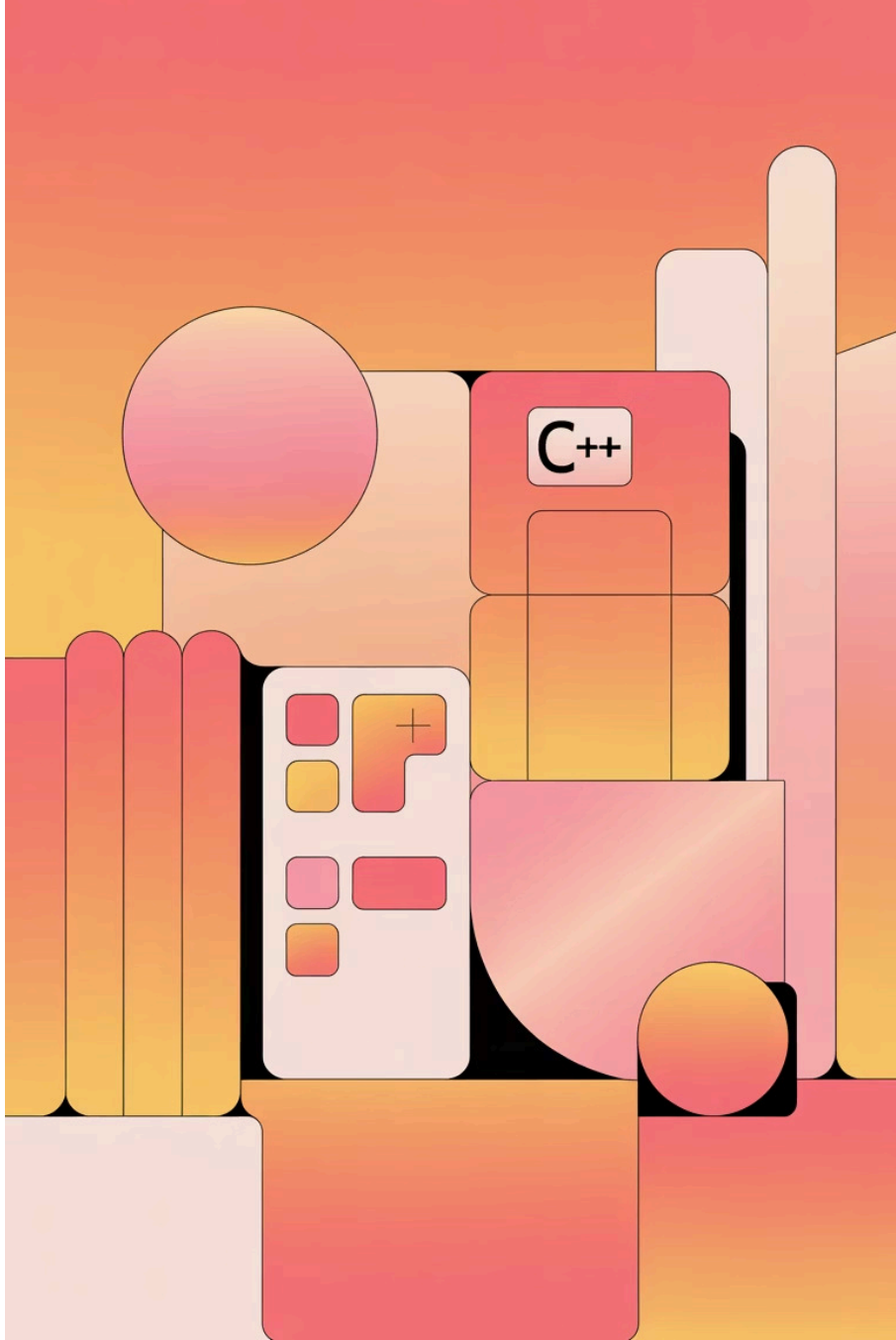
std::ifstream / std::ofstream

文件资源

std::jthread

线程生命周期（C++20）

> C++ 标准库广泛使用 RAII 来封装易错的资源管理。



RAII 与异常安全

- 异常是资源泄漏最常见的来源
- 传统代码需要覆盖所有返回路径
- RAII 保证异常不会跳过析构
- 不需要 try / finally
- 不需要 GC

> RAII 让异常不再是资源管理的特殊情况。

对比：try / finally 的方式

try / finally 方式 (C#/Java)

```
FileStream fs = null;
try
{
    fs = new FileStream("test.txt", FileMode.Open);
    // 使用fs
    // ...
}
finally
{
    if (fs != null)
    {
        fs.Close();   // 一定会执行
    }
}
```

RAII 方式 (C++)

标准库方式：

```
std::ifstream ifs("test.txt");
if (ifs) {
    // 使用 ifs
    // ...
}
```

C 风格文件的 RAII 封装：

```
auto deleter = [](FILE* f) {
    if (f) std::fclose(f);
};

std::unique_ptr fs(
    std::fopen("test.txt", "r"),
    deleter
);

// 使用 fs
```

> try / finally 是人为保证，RAII 是编译器保证。

工程中使用RAII

优先使用智能指针

尽量不要使用裸new和delete，而是选择std::make_unique()和std::make_shared()来创建对象。这能确保资源在超出作用域时自动释放。

Unique Pointer 高性能

std::unique_ptr在性能上与裸指针T*基本持平，几乎没有运行时开销，是独占所有权场景的首选。

Shared Pointer 用于共享

在需要共享所有权的特殊情况下，使用std::make_shared()。它通过引用计数管理资源生命周期，但会带来不少的性能开销。

定制 Deleter 获得更大的灵活性

对于文件句柄、网络连接、锁等内存之外的资源，用定制了deleter的std::unique_ptr或者std::shared_ptr进行管理，统一资源释放逻辑。

必要时实现自定义 RAII 类

如果以上方法满足不了释放资源的需求，那么自己实现RAII类。确保在构造函数中获取资源，在析构函数中释放资源。

> RAII 不仅仅是语言特性，更是一种资源管理的设计哲学，贯穿现代 C++ 工程实践。

RAII使用实例 - 原方案

> `resbuf` 是 AutoCAD 用来表示“变长、异构数据列表”的动态链表结构，
> 通常由 ObjectARX 在堆上分配，必须调用acutRelRb释放。
>
> RAII以前的做法是手工释放，示例代码如下：

```
resbuf* rb = nullptr;

try {
    rb = acutBuildList(
        RTSHORT, 1,
        RTSTR, _T("Hello"),
        RTNONE
    );

    // 使用 rb
    // 任何这里抛出的异常，都必须被兜住
    // ...

    if (/*提前退出条件*/) {
        acutRelRb(rb);
        rb = nullptr;
        return;
    }
}
catch (...) {
    if (rb) {
        acutRelRb(rb);   // 手动释放（异常路径）
        rb = nullptr;
    }
    // 处理异常
}

// 正常路径也必须释放
acutRelRb(rb);
rb = nullptr;
return 0;
```

> 靠人的小心翼翼，一不小心就会漏掉释放资源。

RAII使用实例 - 定制deleter

> 定制智能指针的deleter，调用acutRelRb。

```
// 定制deleter
auto deleter = [](resbuf* p) {
    if (p) acutRelRb(p);
};

// 不管如何离开，resbuf都会被释放
std::unique_ptr<resbuf, decltype(deleter)> rb(
    acutBuildList(
        RTSHORT, 1,
        RTSTR, _T("Hello"),
        RTNONE
    ),
    deleter
);

try {
    if (/*提前退出条件*/) {
        return 0;
    }
}
catch (...) {
    // 处理异常
}

// 正常退出路径
return 0;
```

> 还有更简洁写法

RAII使用实例 - 定制RAII类

> 定制一个RAII类，封装资源管理，独占所有权方式。可以移动不可拷贝。

```
class RbHolder {
public:
    explicit RbHolder(resbuf* p) : rb(p) {}
    ~RbHolder() {
        if (rb) acutRelRb(rb);
    }

    resbuf* get() const { return rb; }

    RbHolder(const RbHolder&) = delete;
    RbHolder& operator=(RbHolder&&) = delete;

    RbHolder(RbHolder&& other) noexcept : rb(other.rb) {
        other.rb = nullptr;
    }

    RbHolder& operator=(RbHolder&& other) noexcept {
        if (this != &other) {
            rb = other.rb;
            other.rb = nullptr;
        }
        return *this;
    }

private:
    resbuf* rb;
};
```

```
int main()
{
    // 不管如何离开，resbuf都会被释放
    RbHolder rb(acutBuildList(
        RTSHORT, 1,
        RTSTR, _T("Hello"),
        RTNONE
    ));

    try {
        if (/*提前退出条件*/) {
            return 0;
        }
    }
    catch (...) {
        // 处理异常
    }

    // 正常退出路径
    return 0;
}
```

> 编译器保证RbHolder对象一定在最后调用acutRelRb以释放resbuf。

本篇总结：RAII的价值

RAII 不只是技巧，而是 C++ 实现资源（内存、文件句柄、连接等）不泄漏的核心保障。

- 语言标准保障
- 不依赖GC
- 异常情况下可靠释放资源
- 所有权清晰

下一篇我们重点关注

模板元编程与constexpr —— 「编译期计算实现零成本抽象」

我们将介绍如何利用模板元编程与 constexpr， 把计算前移到编译期，实现真正的零成本抽象。